**Number of bytes allocated for type of array × Number of elements**

## Initialization

An array can be initialized at the time of its declaration. Unless an array is initialized, all the array elements contain garbage values.

Syntax:

<datatype> <array_name> [size] = {value-1,value-2,…… ,value-n};

Example

int age[5]={19,21,16,1,50};

In the above example, the array name is 'age' whose size is 5. In this case, the first element 19 is stored in age[0], the second element 21 is stored in age[1] and so on as shown in figure 12.1

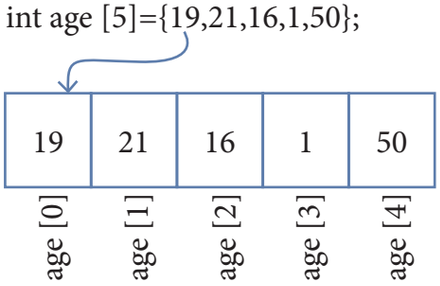



Figure 12.1

While declaring and initializing values in an array, the values should be given within the curly braces ie. { ….. }

The size of an array may be optional when the array is initialized during declaration.

Example:

int age[]={ 19,21,16,1,50};

In the above initialization, the size of the array is not specified directly in the declaration with initialization. So, the size is determined by compiler which depends on the total number of values. In this case, the size of the array is five.

More examples of array initialization:

float x[5] = {5.6, 5.7, 5.8, 5.9, 6.1};

char vowel[6] = {'a', 'e', 'i', 'o', 'u', '\0'};

## Accepting values to an array during run time :

Multiple assignment statements are required to insert values to the cells of the array during runtime. The for loop is ideally suited for iterating through the array elements.

**// Input values while execution**

```
#include <iostream>
using namespace std;
int main()
{
        int num[5];
        for(int i=0; i<5; i++)
        {
                cout<< "\n Enter value " << i+1 << "= ";
                cin>>num[i];
        }
}
```

In the above program, a for loop has been constructed to execute the statements within the loop for 5 times. During each iteration of the loop, *cout* statement prompts you to "Enter value ……." and *cin* gets the value and stores it in num[i];

The following table shows the execution of the above code block.



Chapter 12 Page 206-227.indd 208 3/24/2020 9:22:23 AM